

Trabajo Práctico Integrador

Programación 1

Gestión de Datos de Países en Python

Alumno: Kenneth Lindner

Comisión: 18

Tutor: Flor Camila Gubiotti

Fecha: 28/6/2026

Indice

Indice.....	2
1. Marco Teórico.....	3
1.1 Listas.....	3
1.2 Diccionarios.....	3
1.3 Funciones.....	3
1.4 Estructuras condicionales.....	3
1.5 Estructuras repetitivas.....	4
1.6 Ordenamientos.....	4
1.7 Estadísticas básicas.....	4
1.8 Archivos CSV.....	4
2. Arquitectura.....	5
2.1 Organización del Sistema.....	5
2.2 Diagrama de Flujo del Programa.....	5
2.3 Decisiones del Diseño.....	6
2.4 Capturas.....	6
3. Dificultades y Conclusiones.....	8
3.1 Dificultades Encontradas.....	8
3.2 Conclusiones.....	8
4. Bibliografía.....	8
5. Links.....	8

1. Marco Teórico

1.1 Listas

Una lista en Python es una estructura de datos ordenada y mutable que permite almacenar una colección de elementos. En nuestro proyecto, utilizamos una lista para almacenar el conjunto de países cargados desde el archivo CSV. Cada elemento de la lista es un diccionario que representa un país.

Las listas permiten agregar, eliminar y acceder a elementos por índice, lo cual resulta fundamental para las operaciones de alta y consulta del sistema.

1.2 Diccionarios

Un diccionario es una estructura de datos que almacena pares clave-valor. A diferencia de las listas, el acceso a los datos se realiza mediante claves (strings) en lugar de índices numéricos, lo que mejora la legibilidad del código.

En nuestro proyecto, cada país se representa como un diccionario con las claves 'nombre', 'poblacion', 'superficie' y 'continente'. Esto permite acceder a los atributos de forma clara y directa (por ejemplo, `pais["nombre"]`).

1.3 Funciones

Las funciones son bloques de código reutilizable que realizan una tarea específica. Permiten modularizar el programa, facilitando la lectura, el mantenimiento y las pruebas.

Aplicamos el principio de *una función = una responsabilidad*, separando el código en módulos:

- 'archivo.py': lectura y escritura del CSV
- 'validaciones.py': validación de entradas del usuario
- 'países.py': lógica de negocio (CRUD, filtros, ordenamiento, estadísticas)
- 'main.py': menú principal y flujo del programa

1.4 Estructuras condicionales

Las estructuras condicionales ('if', 'elif', 'else') permiten ejecutar diferentes bloques de código según se cumplan o no determinadas condiciones. En nuestro proyecto las utilizamos para:

- Dirigir el flujo del menú según la opción elegida
- Validar entradas del usuario
- Aplicar filtros sobre los datos

1.5 Estructuras repetitivas

Los bucles ('while', 'for') permiten ejecutar un bloque de código múltiples veces. Utilizamos:

- 'while True' para mantener el menú activo hasta que el usuario elija salir
- 'while' en las validaciones para solicitar datos hasta obtener un valor correcto
- 'for' para recorrer la lista de países al filtrar, buscar u obtener estadísticas

1.6 Ordenamientos

El ordenamiento consiste en reorganizar los elementos de una colección según un criterio.

Python provee la función 'sorted()' que permite ordenar cualquier iterable a partir de una función clave ('key') y un sentido ('reverse').

En nuestro sistema el usuario puede ordenar por nombre, población o superficie, tanto ascendente como descendente.

```
campos = {"1": "nombre", "2": "poblacion", "3": "superficie"}
campo = campos[campo_opcion]
descendente = orden == "desc"

ordenados = sorted(paises, key=lambda p: p[campo] if isinstance(p[campo], int) else p[campo].lower(), reverse=descendente)
mostrar_lista(ordenados, "ordenamiento")
```

1.7 Estadísticas básicas

Las estadísticas permiten obtener indicadores clave a partir de un conjunto de datos.

Implementamos:

- **Máximo y mínimo:** usando 'max()' y 'min()' con una función clave
- **Promedio:** suma total dividida por la cantidad de elementos
- **Conteo por categoría:** usando un diccionario acumulador para contar países por continente

1.8 Archivos CSV

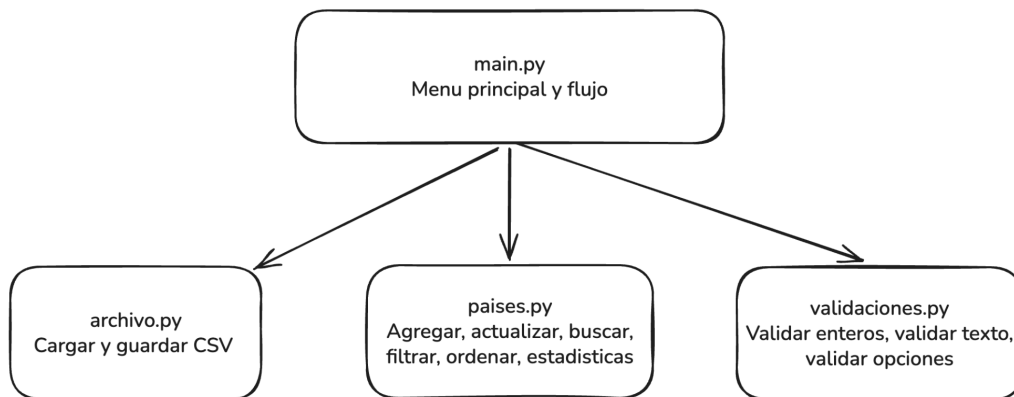
CSV (Comma-Separated Values) es un formato de texto plano para almacenar datos tabulares. Cada línea representa un registro y los campos se separan por comas.

Para leer el archivo utilizamos 'open()' junto con 'split(",")' para separar cada línea en sus campos, y convertimos los valores numéricos con 'int()'. Para escribir, reconstruimos cada línea con f-strings y las volcamos al archivo.

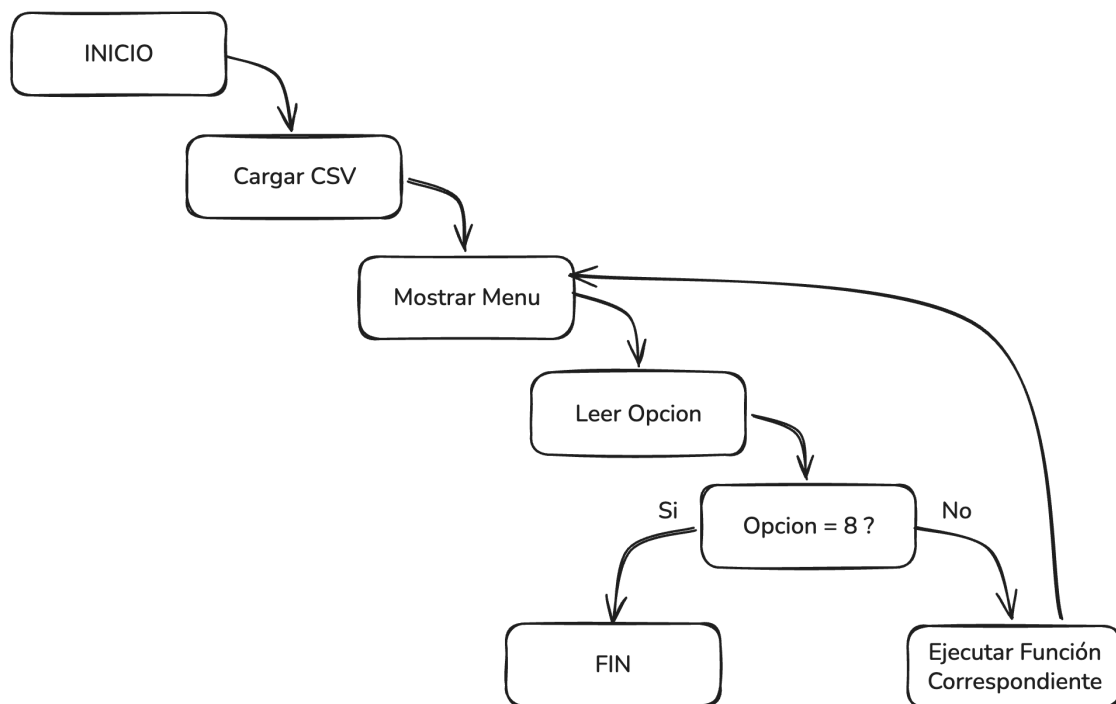
2. Arquitectura

2.1 Organización del Sistema

El sistema se dividió en 4 módulos con responsabilidades separadas:



2.2 Diagrama de Flujo del Programa



2.3 Decisiones del Diseño

- **Persistencia inmediata:** Se guarda el CSV tras cada modificación para evitar pérdida de datos ante cierres inesperados.
- **Búsqueda case-insensitive:** Se convierten los strings a minúsculas antes de comparar, mejorando la experiencia de usuario.
- **Validación en loop:** Las funciones de validación repiten la solicitud hasta obtener un dato correcto, evitando crashes por inputs inválidos.
- **Verificación de duplicados:** Al agregar un país, se verifica que no exista otro con el mismo nombre.

2.4 Capturas

```
kenneth.lindner@HWTN47PWNV TPI % /opt/homebrew/bin/python3 ,
Iniciando sesión. Bienvenido!

INFO: Se cargaron 12 países desde 'países.csv'

===== GESTIÓN DE PAISES =====
1. Agregar país
2. Actualizar país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Mostrar todos los países
8. Salir
=====
Seleccione una opción: █
```

Imagen 1: Inicio del Programa

```
-----
Seleccione una opción: 7

Nombre                Población    Superficie Continente
-----
Argentina             45,376,763   2,780,400 América
Brasil                213,993,437   8,515,767 América
Japón                 125,800,000   377,975 Asia
Alemania              83,149,300    357,022 Europa
Nigeria              223,804,632   923,768 África
Australia             26,473,055    7,692,024 Oceanía
México               128,901,000   1,964,375 América
Francia               67,390,000    643,801 Europa
India                 1,428,627,663 3,287,263 Asia
Egipto               104,258,327   1,001,450 África
Chile                 19,000,000    800,000 América
Kenia                 80,000,000    568,000 África

Total: 12 país(es)
```

Imagen 2: Listar todos los Países

Seleccione una opción: 6

--- Estadísticas ---

Mayor población: India (1,428,627,663)

Menor población: Chile (19,000,000)

Promedio de población: 212,231,181

Promedio de superficie: 2,409,320 km²

Cantidad de países por continente:

América: 4

Asia: 2

Europa: 2

África: 3

Oceanía: 1

Imagen 3: Estadísticas de Países

===== GESTIÓN DE PAISES =====

1. Agregar país
2. Actualizar país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Mostrar todos los países
8. Salir

=====

Seleccione una opción: 1

Nombre del país: Uruguay

Población: 3386544

Superficie (km²): 176215

Continente: América

País 'Uruguay' agregado exitosamente

Imagen 4: Agregar Nuevo País

3. Dificultades y Conclusiones

3.1 Dificultades Encontradas

Durante el desarrollo del proyecto surgieron diversos desafíos técnicos que requirieron ajustes en la implementación. Uno de los principales inconvenientes fue el manejo de los datos provenientes del archivo CSV, ya que algunos registros podrían contener caracteres especiales o formatos inesperados que ocasionan errores durante la lectura. Además, como el archivo se procesó utilizando **open()** y **split()** para separar los campos por comas, fue necesario asegurarse de que los nombres ingresados por el usuario no incluyeran comas, ya que esto rompería la estructura del archivo CSV y provocaría una lectura incorrecta de los datos.

También se implementaron validaciones para verificar que los valores numéricos se encontraran dentro de los rangos permitidos y evitar que datos inválidos afectaran el funcionamiento del programa. Finalmente, se presentaron dificultades al momento de formatear la tabla de salida para que la información se mostrara de manera clara y alineada, las cuales se resolvieron mediante un mejor manejo del espaciado y la presentación de los datos.

3.2 Conclusiones

El desarrollo de este proyecto permitió reforzar conceptos fundamentales de programación, especialmente la importancia de modularizar el código para facilitar su mantenimiento, comprensión y reutilización. Asimismo, se evidenció que el uso adecuado de estructuras de datos simplifica la organización y manipulación de la información, haciendo que el programa sea más eficiente y fácil de ampliar. Finalmente, se comprendió la relevancia de validar las entradas y manejar correctamente los posibles errores, ya que esto incrementa la confiabilidad de la aplicación y mejora la experiencia del usuario.

4. Bibliografía

- Material provisto por la Cátedra - Programación 1 - UTN
- Documentación Oficial de Python 3 - Modulo CSV, Estructuras de Datos

5. Links

- **Repo:** <https://github.com/klindner-utn/UTN-TUPaDProgramacion1/tree/main/TPI>
- **Video:**  TPI-Programacion1.mov